

HARDWARE MATRIX MULTIPLICATION USING VITIS HLS

Experiment No.: 3 (in FPGA)

Title: Design and Implementation of a Matrix Multiplication Accelerator using Vitis HLS on PYNQ-Z2 FPGA

Prepared By:

NAME: Chinmay N. Yalawatti

USN: 01FE24BEC258

Semester: 5th sem

Branch: E&C (E-div)

College: KLE Technological University BVB campus

Aim

To design, synthesize, implement and verify a hardware matrix multiplication accelerator using Vitis High-Level Synthesis and deploy the generated hardware IP on the PYNQ-Z2 FPGA platform.

Problem Statement

Design an HLS accelerator that performs multiplication of two 4×4 matrices.

[$C = A \cdot B$]

where:

- A = Input Matrix

- B = Input Matrix
- C = Output Matrix

The accelerator should be synthesized in Vitis HLS and exported as an IP for integration into the PYNQ-Z2 overlay.

Theory

Matrix multiplication is one of the most important operations in:

- Artificial Intelligence
- Image Processing
- Digital Signal Processing
- Scientific Computing

Instead of executing multiplication on a CPU, FPGA hardware can perform operations in parallel, providing higher throughput and lower execution time.

Vitis HLS converts C/C++ algorithms into synthesizable RTL (Verilog/VHDL). The generated IP is imported into Vivado, connected to the Zynq Processing System, and programmed onto the FPGA.

Algorithm

1. Read Matrix A.
2. Read Matrix B.
3. Initialize output matrix.
4. Multiply corresponding row and column elements.
5. Store the result.
6. Return output matrix.
7. Verify using testbench.
8. Synthesize into RTL.
9. Package as IP.
10. Integrate with Zynq.
11. Generate Bitstream.
12. Program FPGA.

Hardware Requirements

- PYNQ-Z2 FPGA Board
- USB Cable
- PC

Software

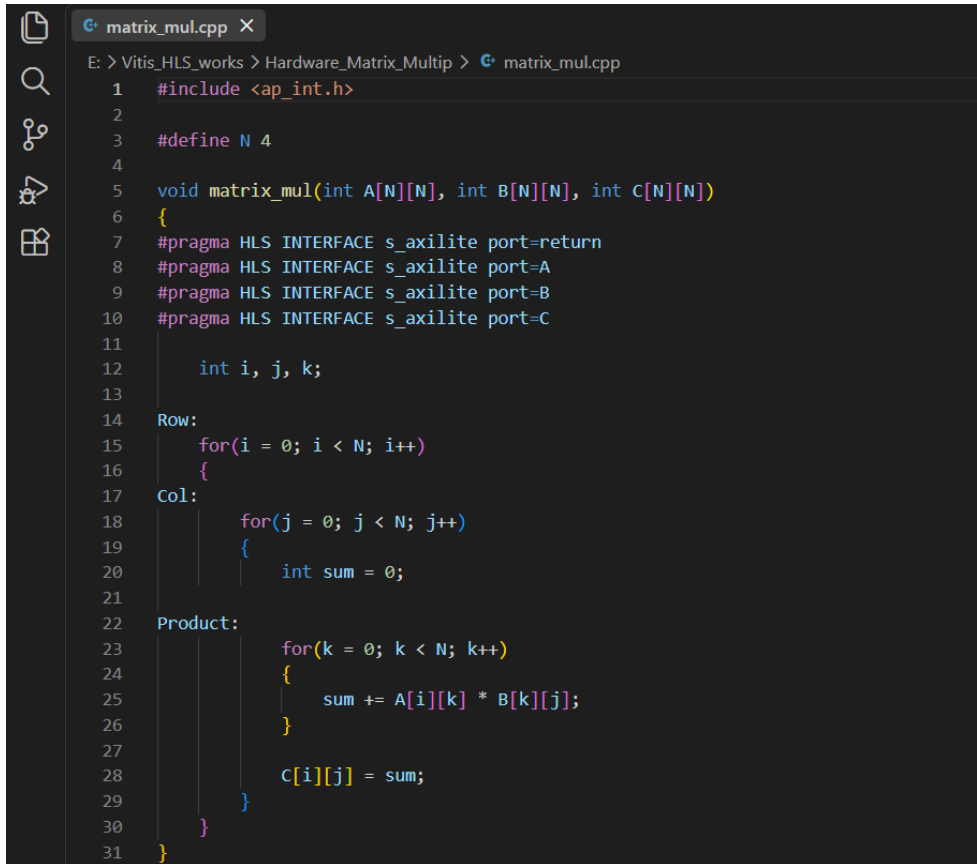
- Vitis Unified IDE 2026.1
- Vivado 2026.1

Target FPGA

xc7z020clg400-1

HLS Source Code

matrix_mul.cpp



```
1  #include <ap_int.h>
2
3  #define N 4
4
5  void matrix_mul(int A[N][N], int B[N][N], int C[N][N])
6  {
7      #pragma HLS INTERFACE s_axilite port=return
8      #pragma HLS INTERFACE s_axilite port=A
9      #pragma HLS INTERFACE s_axilite port=B
10     #pragma HLS INTERFACE s_axilite port=C
11
12     int i, j, k;
13
14     Row:
15     for(i = 0; i < N; i++)
16     {
17         Col:
18         for(j = 0; j < N; j++)
19         {
20             int sum = 0;
21
22             Product:
23             for(k = 0; k < N; k++)
24             {
25                 sum += A[i][k] * B[k][j];
26             }
27
28             C[i][j] = sum;
29         }
30     }
31 }
```

Figure 6.1 : Matrix Multiplication HLS Source Code

Testbench

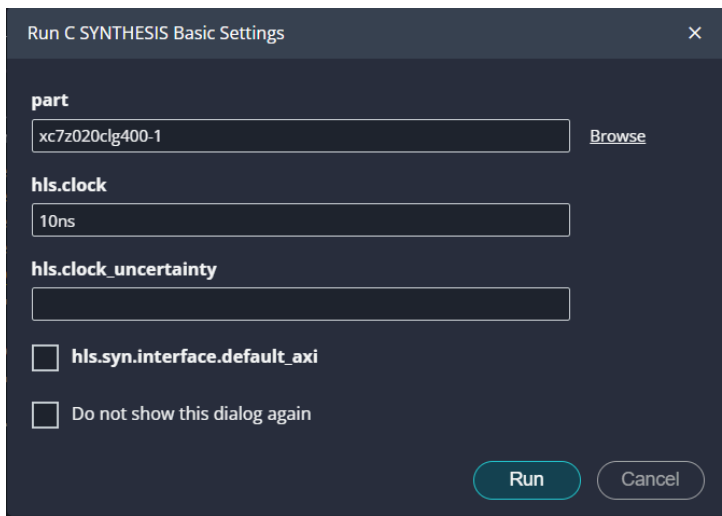
matrix_mul_tb.cpp

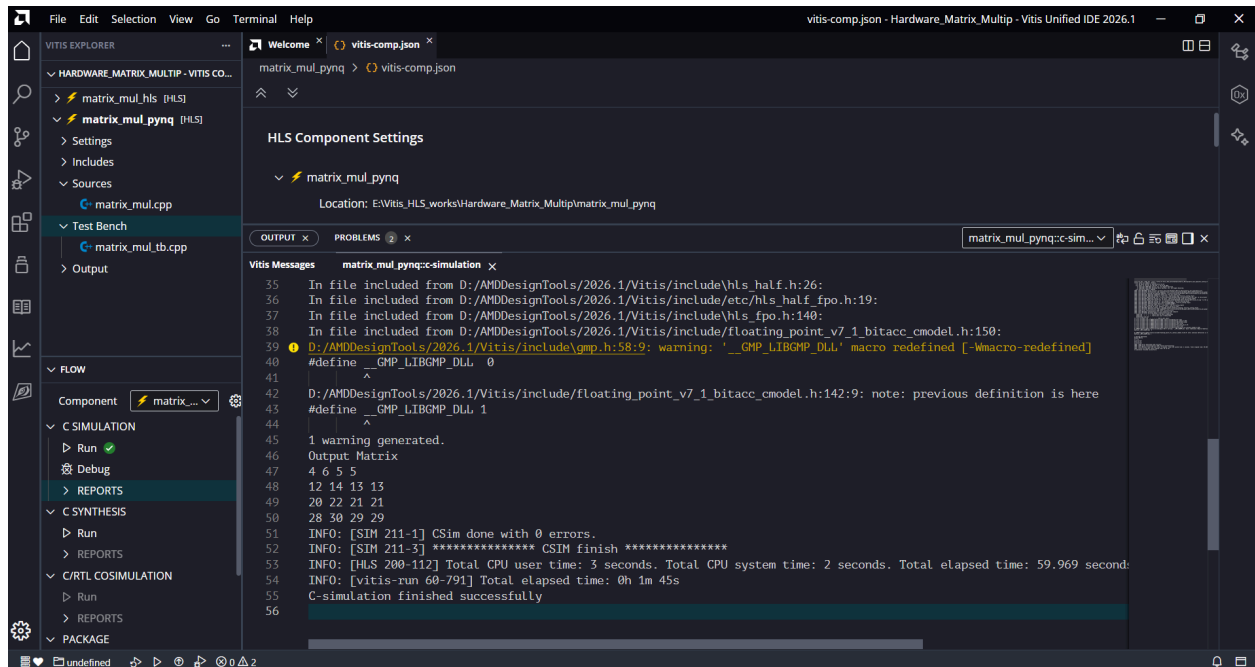
```
E: > Vitis_HLS_works > Hardware_Matrix_Multip > matrix_mul_tb.cpp
1  #include <iostream>
2  using namespace std;
3
4  #define N 4
5
6  void matrix_mul(int A[N][N], int B[N][N], int C[N][N]);
7
8  int main()
9  {
10     int A[N][N] =
11     {
12         {1,2,3,4},
13         {5,6,7,8},
14         {9,10,11,12},
15         {13,14,15,16}
16     };
17
18     int B[N][N] =
19     {
20         {1,0,0,1},
21         {0,1,1,0},
22         {1,0,1,0},
23         {0,1,0,1}
24     };
25
26     int C[N][N];
27
28     matrix_mul(A,B,C);
29
30     cout<<"Output Matrix"<<endl;
31
32     for(int i=0;i<N;i++)
33     {
34         for(int j=0;j<N;j++)
35         {
36             cout<<C[i][j]<<" ";
37         }
38     }
39
40     return 0;
41 }
```

```
E: > Vitis_HLS_works > Hardware_Matrix_Multip > matrix_mul_tb.cpp
9  {
16  };
17
18  int B[N][N] =
19  {
20      {1,0,0,1},
21      {0,1,1,0},
22      {1,0,1,0},
23      {0,1,0,1}
24  };
25
26  int C[N][N];
27
28  matrix_mul(A,B,C);
29
30  cout<<"Output Matrix"<<endl;
31
32  for(int i=0;i<N;i++)
33  {
34      for(int j=0;j<N;j++)
35      {
36          cout<<C[i][j]<<" ";
37      }
38      cout<<endl;
39  }
40
41  return 0;
42 }
```

Figure 7.1 : Testbench Code

C Simulation





The HLS design was verified using the testbench. The generated output matched the expected matrix multiplication result, confirming functional correctness.

C Synthesis

Summary Report: The C algorithm was successfully converted into synthesizable RTL.

File Edit Selection View Go Terminal Help

matrix_mul_pynq - C SYNTHESIS - Hardware_Matrix_Multip - Vitis Unified IDE 2026.1

VITIS EXPLORER

- HARDWARE_MATRIX_MULTIP - VITIS CO...
- matrix_mul_hls (HLS)
- matrix_mul_pynq (HLS)
- Settings
- Includes
- Sources
 - matrix_mul.cpp
- Test Bench
 - matrix_mul_tb.cpp
- Output

FLOW

Component: matrix_...

C SIMULATION

- Run
- Debug
- REPORTS

SYNTHESIS

- Run
- REPORTS
 - Summary
 - Synthesis
 - Function Call Graph
 - Schedule Viewer

Summary Synthesis Report - matrix_mul

General Information

- Estimated Quality of Results
 - Timing Estimate
 - Performance & Resource Estimates
- HW Interfaces
 - S_AXILITE Interfaces
 - S_AXILITE Registers
 - TOP LEVEL CONTROL
- SW I/O Information
 - Top Function Arguments
 - SW-to-HW Mapping
- Pragma Report
 - Implemented Pragas
 - Valid Pragma Syntax

General Information

Version: 2026.1 (Build 6493734 on Jun 16 2026)

Product family: zynq

Target device: xc7z020-clg400-1

Estimated Quality of Results

Timing Estimate

TARGET	WITH UNCERTAINTY	ESTIMATED
10.00 ns	7.30 ns	6.923 ns

Performance & Resource Estimates

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM
matrix_mul (1)	690.000		64		loop auto-rewind	function	
Row_Col	670.000	8	4	16	yes	loop	

Vitis Messages

matrix_mul_pynq::c-simulation

matrix_mul_pynq::synthesis

```
93 INFO: [VHDL 208-304] Generating VHDL RTL for matrix_mul.
94 INFO: [VLOG 209-307] Generating Verilog RTL for matrix_mul.
95 INFO: [HLS 200-789] **** Loop Constraint Status: All loop constraints were satisfied.
96 INFO: [HLS 200-789] **** Estimated Fmax: 144.45 MHz
97 INFO: [HLS 200-112] Total CPU user time: 2 seconds. Total CPU system time: 2 seconds. Total elapsed time: 65.274 seconds
98 INFO: [v++ 60-791] Total elapsed time: 0h 1m 34s
99 Synthesis finished successfully, open report.
100
```

Performance & Resource Estimates

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM
matrix_mul (1)	690.000		64		loop auto-rewind	function	
Row_Col	670.000	8	4	16	yes	loop	

Summary Synthesis Report - matrix_mul

General Information

- Estimated Quality of Results
 - Timing Estimate
 - Performance & Resource Estimates
- HW Interfaces
 - S_AXILITE Interfaces
 - S_AXILITE Registers
 - TOP LEVEL CONTROL
- SW I/O Information
 - Top Function Arguments
 - SW-to-HW Mapping
- Pragma Report
 - Implemented Pragas
 - Valid Pragma Syntax
 - Inferred Pragas
- Bind Op Report
 - User Config Op
- Bind Storage Report
 - User Config Storage

General Information

Version: 2026.1 (Build 6493734 on Jun 16 2026)

Product family: zynq

Target device: xc7z020-clg400-1

Estimated Quality of Results

Timing Estimate

TARGET	WITH UNCERTAINTY	ESTIMATED
10.00 ns	7.30 ns	6.923 ns

Performance & Resource Estimates

MODULES & LOOPS	LATENCY(NS)	ITERATION LATENCY	INTERVAL	TRIP COUNT	PIPELINED	STRUCTURE	BRAM	DSP	FF	LUT	URAM
matrix_mul (1)	690.000		64		loop auto-rewind	function	0	3	799	674	0
Row_Col	670.000	8	4	16	yes	loop					

Welcomevitis-comp.jsonmatrix_mul_pynq - c SYNTHESIS

Synthesis

Summary Synthesis Report - matrix_mul

General Information

Estimated Quality of Results

Timing Estimate

Performance & Resource Estim...

HW Interfaces

S_AXILITE Interfaces

S_AXILITE Registers

TOP LEVEL CONTROL

SW I/O Information

Top Function Arguments

SW-to-HW Mapping

Pragma Report

Implemented Pragmas

Valid Pragma Syntax

Inferred Pragmas

Bind Op Report

User Config Op

Bind Storage Report

User Config Storage

HW Interfaces

S_AXILITE Interfaces

INTERFACE	DATA WIDTH	ADDRESS WIDTH	OFFSET	REGISTER
s_axi_control	32	8	64	0

S_AXILITE Registers

INTERFACE	REGISTER	OFFSET	WIDTH	ACCESS	DESCRIPTION
s_axi_control	CTRL	0x00	32	RW	Control signals
s_axi_control	GIER	0x04	32	RW	Global Interrupt Enable Register
s_axi_control	IP_IER	0x08	32	RW	IP Interrupt Enable Register
s_axi_control	IP_ISR	0x0c	32	RW	IP Interrupt Status Register

TOP LEVEL CONTROL

INTERFACE	TYPE	PORTS
ap_clk	clock	ap_clk
ap_rst_n	reset	ap_rst_n
interrupt	interrupt	interrupt

OUTPUTPROBLEMS 3

matrix_mul_pynq::synt...

Synthesis

Summary Synthesis Report - matrix_mul

General Information

Estimated Quality of Results

Timing Estimate

Performance & Resource Estimates

HW Interfaces

S_AXILITE Interfaces

S_AXILITE Registers

TOP LEVEL CONTROL

SW I/O Information

Top Function Arguments

SW-to-HW Mapping

Bind Op Report

User Config Op

Bind Storage Report

User Config Storage

TOP LEVEL CONTROL

INTERFACE	TYPE	PORTS
ap_clk	clock	ap_clk
ap_rst_n	reset	ap_rst_n
interrupt	interrupt	interrupt
ap_ctrl	ap_ctrl_hs	

SW I/O Information

Top Function Arguments

ARGUMENT	DIRECTION	DATATYPE
A	in	int*
B	in	int*
C	out	int*

SW-to-HW Mapping

ARGUMENT	HW INTERFACE	HW TYPE	HW INFO
A	s_axi_control	memory	name=A offset=64 range=64

OUTPUTPROBLEMS 3

matrix_mul_pynq::synt...

Summary Synthesis Report - matrix_mul

General Information

Argument	HW Interface	HW Type	HW Info
B	in	int*	
C	out	int*	

SW-to-HW Mapping

Argument	HW Interface	HW Type	HW Info
A	s_axi_control	memory	name=A offset=64 range=64
B	s_axi_control	memory	name=B offset=128 range=64
C	s_axi_control	memory	name=C offset=192 range=64

Pragma Report

Implemented Pragas

Type	Options	Location	Function
loop_tripcount	max=4 min=4 avg=4	matrix_mul.cpp:12	matrix_mul
loop_flatten		matrix_mul.cpp:18	matrix_mul

Valid Pragma Syntax

Inferred Pragas

Summary Synthesis Report - matrix_mul

General Information

Bind Op Report

Name	DSP	Pragma	Variable	Op	Impl
matrix_mul (1)	3				
Row_Col (15)					

User Config Op

N/A

Bind Storage Report

Name	Usage	Type	BRAM	URAM	Pragma	Variable
matrix_mul (1)						
Row_Col						

User Config Storage

N/A

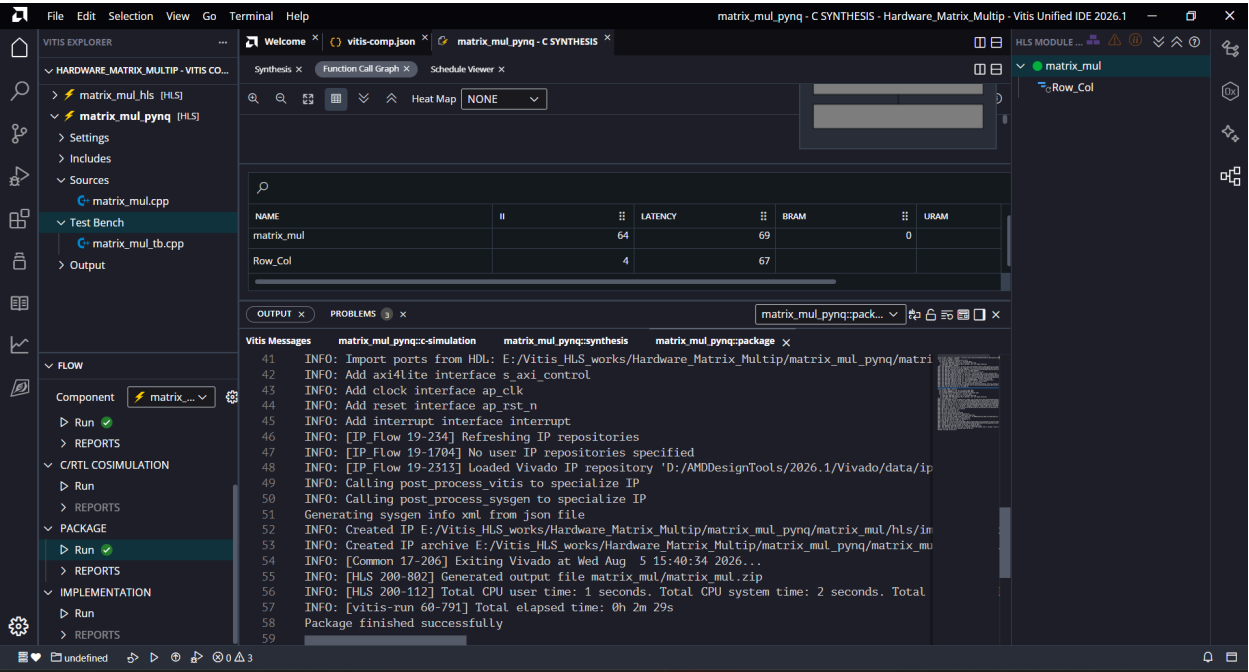
After synthesis:

Resource	Used
Latency	690000
LUT	674

Resource	Used
FF	799
BRAM	0
DSP	3



Function Call Graph



The synthesis report confirms that the matrix multiplication function was successfully optimized and scheduled for hardware implementation.



Vivado Integration

Block Design Screenshot

Hardware_Matrix_Multiplication - [E:/Vitis_HLS_works/Hardware_Matrix_Multiplication/Hardware_Matrix_Multiplication.xpr] - Vivado 2026.1

File Edit Flow Tools Reports Window Layout View Help Quick Access Ready

FLOW NAVIGATOR

- Project Manager
 - Settings
 - Add Sources
 - Language Templates
 - IP Catalog
 - IP Integrator
 - Create Block Design
 - Open Block Design
 - Generate Block Design
 - Simulation
 - Run Simulation
 - RTL Analysis
 - Run Linter
 - Open Elaborated Design
 - Synthesis
 - Run Synthesis
 - Open Synthesized Design
 - Implementation
 - Run Implementation
 - Open Implemented Design

Project Manager - Hardware_Matrix_Multiplication

Sources

- Design Sources
- Constraints
- Simulation Sources
 - sim_1 (active)

Hierarchy Libraries Compile Order

Properties

Select an object to see properties

Project Summary

Overview Dashboard

Settings

Project name: Hardware_Matrix_Multiplication

Project location: E:/Vitis_HLS_works/Hardware_Matrix_Multiplication

Product family: Zynq-7000

Project part: xc7z020cig400-1

Top module name: Not defined

Target language: Verilog

Simulator language: Mixed

Target Simulator: Vivado Simulator

Synthesis Implementation

Tcl Console Messages Log Reports Design Runs

Name	Constraints	Status	WNS	TNS	WHS	THS	WBSS	TPWS	Total Power	Failed Routes	Methodology	RQA Score	QoR Suggestions	LUT	FF	BRAM	URAM	DSP	Start	Elapsed	Run Strate
synth_1	constrs_1	Not started																			Vivado Syn
impl_1	constrs_1	Not started																			Vivado Im

Hardware_Matrix_Multiplication - [E:/Vitis_HLS_works/Hardware_Matrix_Multiplication/Hardware_Matrix_Multiplication.xpr] - Vivado 2026.1

File Edit Flow Tools Reports Window Layout View Help Quick Access Ready

FLOW NAVIGATOR

- Project Manager
 - Settings
 - Add Sources
 - Language Templates
 - IP Catalog
 - IP Integrator
 - Create Block Design
 - Open Block Design
 - Generate Block Design
 - Simulation
 - Run Simulation
 - RTL Analysis
 - Run Linter
 - Open Elaborated Design
 - Synthesis
 - Run Synthesis
 - Open Synthesized Design
 - Implementation
 - Run Implementation
 - Open Implemented Design

Block Design - design_1

Sources

- design_1
 - External Interfaces
 - Interface Connections
 - processing_system7_0 (ZYNQ7 Processing System5.5)

IP Properties

Matrix_mul

Version: 1.0 (Rev. 2114728178)

Interfaces: AXI4

Description: An IP generated by Vitis HLS

IP Catalog (3)

Cores

Search

User Repository (e:/Vitis_HLS_works/Ha)

VITIS HLS IP

Matrix_mul

Vivado Repository

Details

Name: Matrix_mul

Version: 1.0 (Rev. 2114728178)

Diagram

Address Editor Address Map IP Catalog (2)

Default View

processing_system7_0

M_AXI_GP0_ACLK

ZYNQ

DDR

FIXED_IO

M_AXI_GP0

FCLK_CLK0

FCLK_RESET_N

ZYNQ7 Processing System

Tcl Console Messages Log Reports Design Runs

```

open_bd_design: Time (s): cpu = 00:00:07 ; elapsed = 00:00:29 . Memory (MB): peak = 1741.840 ; gain = 0.000
set_property location {1 142 71} [get_bd_cells processing_system7_0]
set_property ip_repo_paths {E:/Vitis_HLS_works/Hardware_Matrix_Multip/matrix_mul_pynq/matrix_mul/hls/impl/ip} [current_project]
update_ip_catalog
INFO: [IP_Flow 19-234] Refreshing IP repositories
INFO: [IP_Flow 19-1700] Loaded user IP repository 'E:/Vitis_HLS_works/Hardware_Matrix_Multip/matrix_mul_pynq/matrix_mul/hls/impl/ip'.
1
get_property ip_repo_paths [current_project]
E:/Vitis_HLS_works/Hardware_Matrix_Multip/matrix_mul_pynq/matrix_mul/hls/impl/ip
  
```

Type a Tcl command here

Hardware_Matrix_Multiplication - [E:/Vitis_HLS_works/Hardware_Matrix_Multiplication/Hardware_Matrix_Multiplication.xpr] - Vivado 2026.1

File Edit Flow Tools Reports Window Layout View Help Quick Access Ready

FLOW NAVIGATOR

- Project Manager
 - Settings
 - Add Sources
 - Language Templates
 - IP Catalog
 - IP Integrator**
 - Create Block Design
 - Open Block Design
 - Generate Block Design
- Simulation
 - Run Simulation
- RTL Analysis
 - Run Linter
 - Open Elaborated Design
- Synthesis
 - Run Synthesis
 - Open Synthesized Design
- Implementation
 - Run Implementation
 - Open Implemented Design

Block Design - design_1

Sources Design Signals

design_1

- External Interfaces
- Interface Connections
- Nets
- axi_smc (AXI SmartConnect 1.0)
- matrix_mul_0 (Matrix_mul1.0)
- processing_system7_0 (ZYNQ7 Processing System)

IP Catalog (3)

Search

Cores

Inte

Name

User Repository (e:/Vitis_HLS)

VITIS HLS IP

Matrix_mul

Vivado Repository

Automotive and Industri

DSP and Math

Embedded Processors

Details

Select an IP or Interface or Repository to see details

Block Interface Properties

s_axi_control

Name

s_axi_control

Mode

SLAVE

Connection

axi_smc_M00_AXI

General Properties

Tcl Console

Messages Log Reports Design Runs

INFO: [IP_Flow 19-234] Refreshing IP repositories

INFO: [IP_Flow 19-1700] Loaded user IP repository 'e:/Vitis_HLS_works/Hardware_Matrix_Multip/matrix_mul_pyng/matrix_mul_hls/impl/ip'.

1

get_property ip_repo_paths [current_project]

E:/Vitis_HLS_works/Hardware_Matrix_Multip/matrix_mul_pyng/matrix_mul_hls/impl/ip

Type a Tcl command here

Diagram Address Editor Address Map IP Catalog (2)

Default View

processing_system7_0

matrix_mul_0

axi_smc

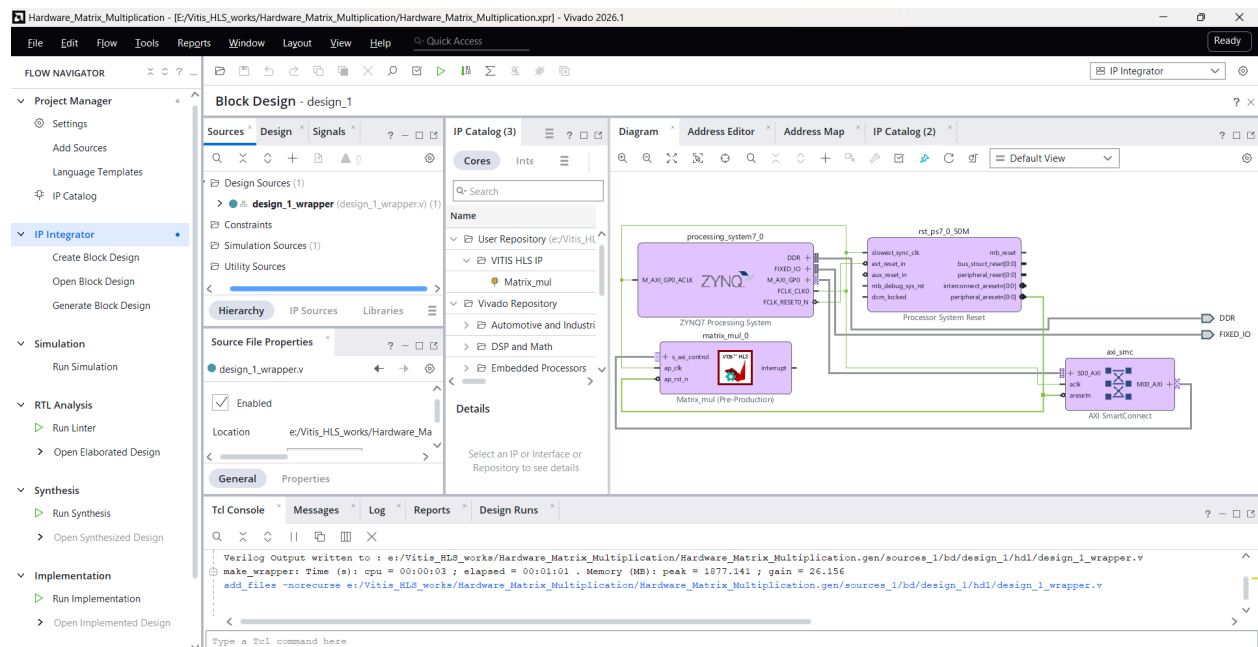
Matrix mul (Pre-Production)

AXI SmartConnect

Validate Design

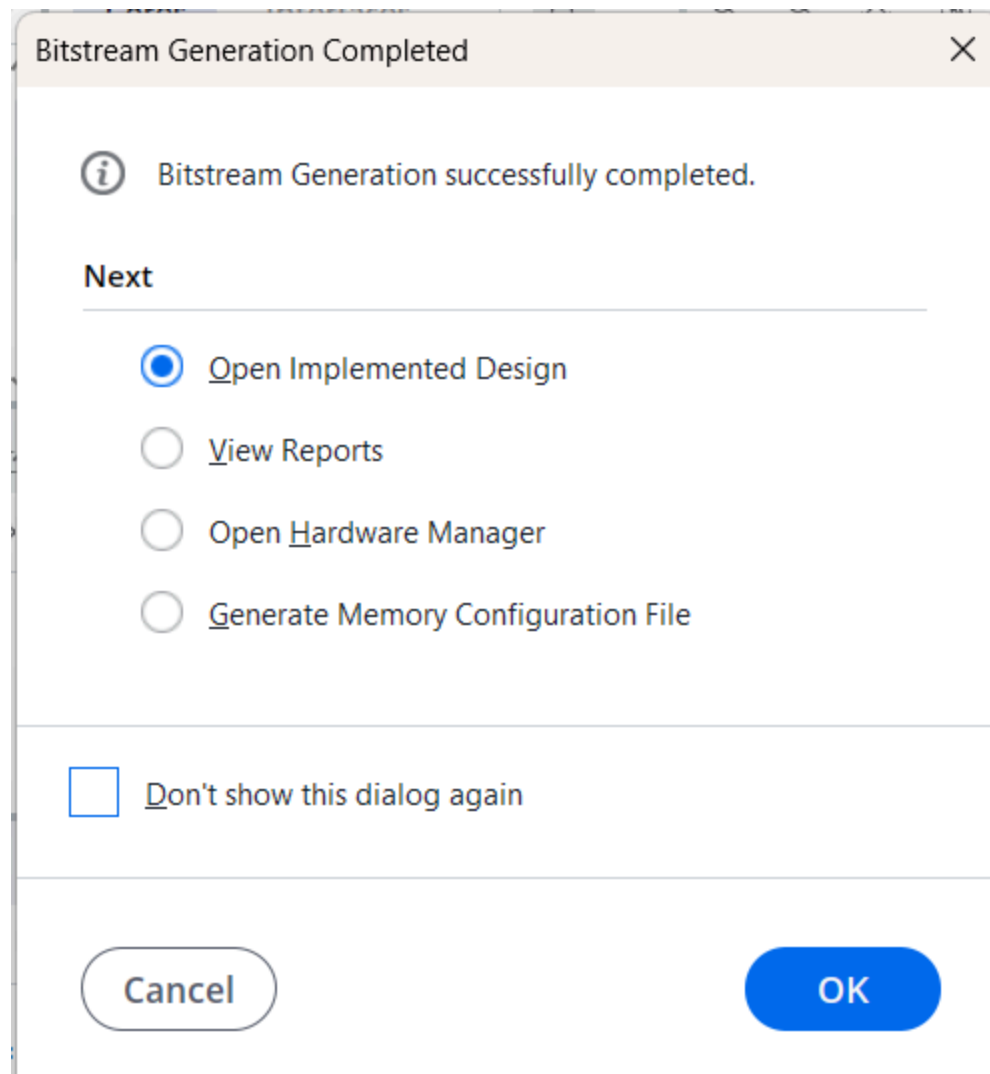
Validation successful. There are no errors or critical warnings in this design.

OK



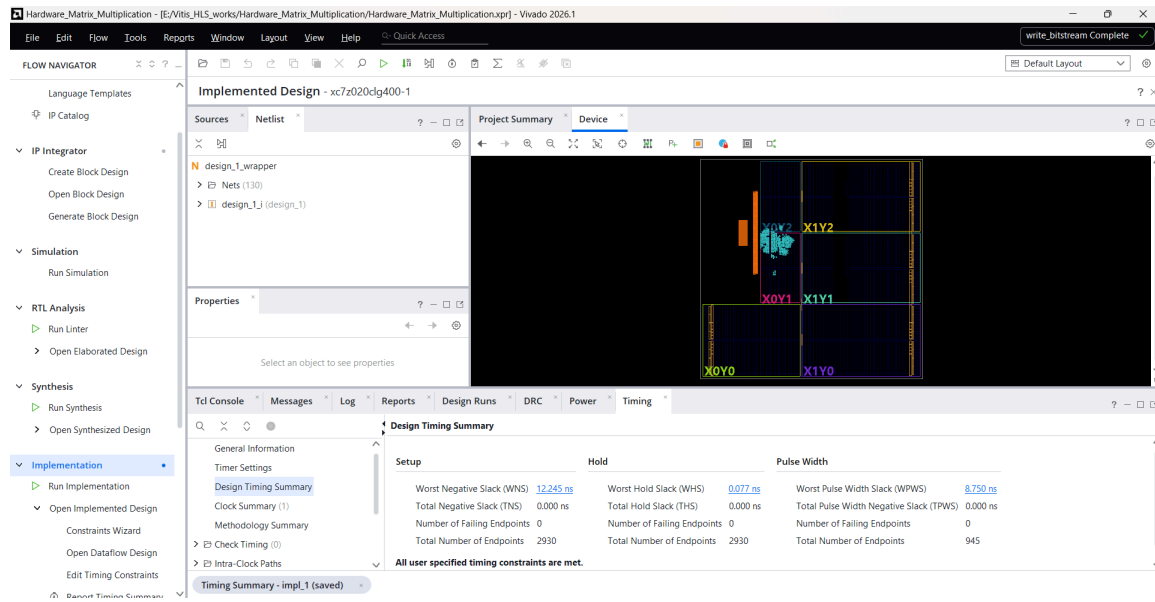
The packaged HLS IP was successfully integrated with the Zynq Processing System through the AXI interface.

Bitstream Generation



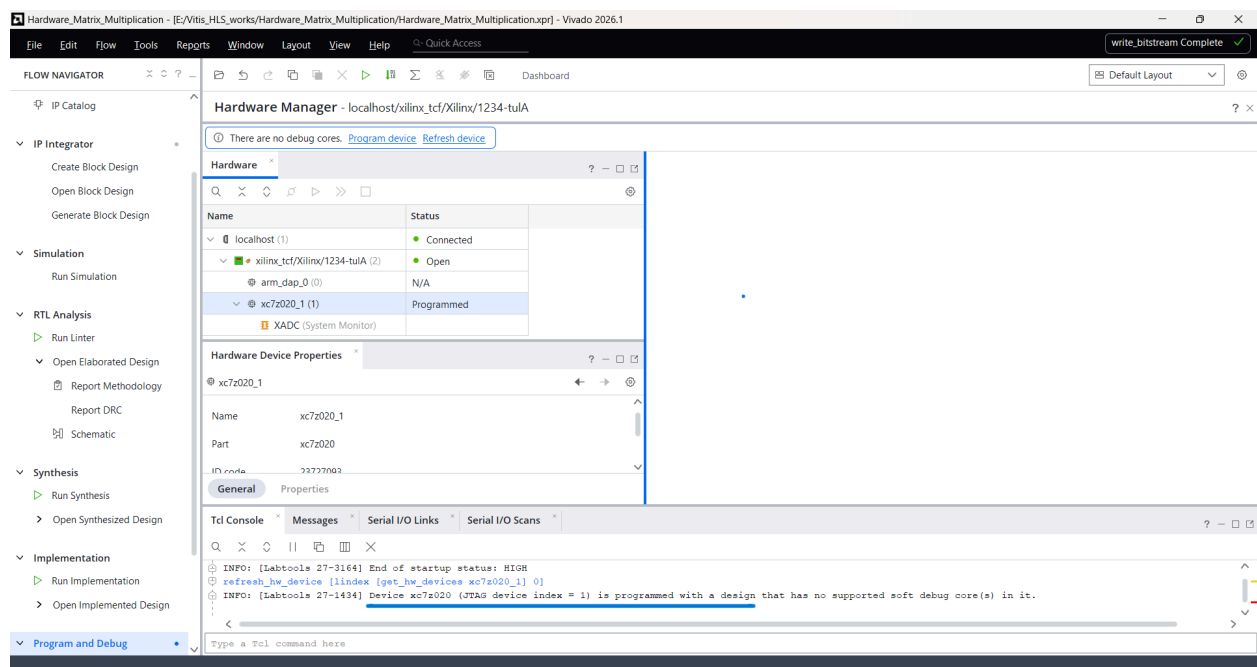
Implementation Complete

Bitstream Generated

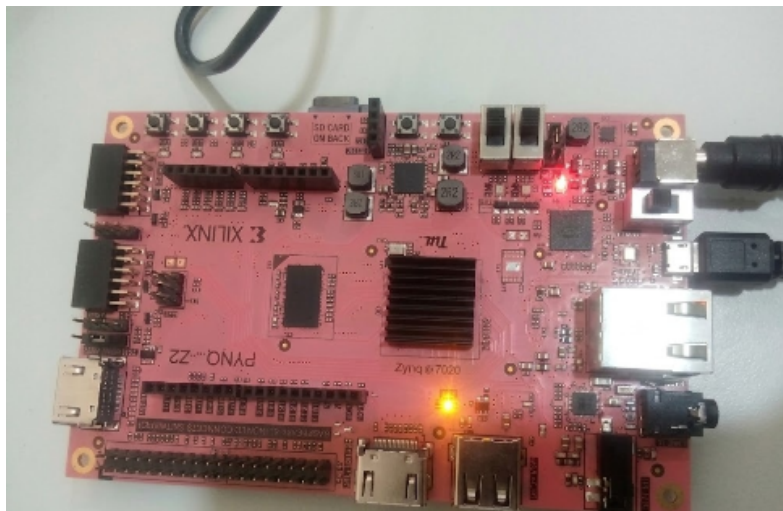
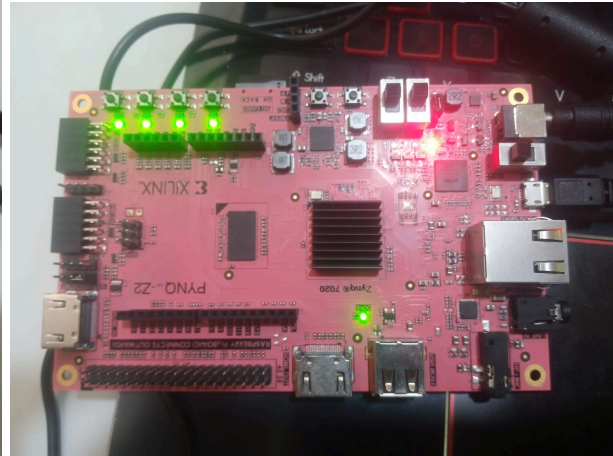


After synthesis and implementation, Vivado generated the FPGA programming bitstream successfully without timing violations.

Hardware Programming



PYNQ Board Photograph:



The orange LED (Done LED) shows the program execution.

Results

Experiment Successfully Completed. The generated bitstream was successfully downloaded to the PYNQ-Z2 FPGA through JTAG using Vivado Hardware Manager.

The Hardware Manager confirmed that the FPGA device was programmed successfully.

Advantages

- Parallel computation
 - Faster execution
 - Low latency
 - Efficient hardware utilization
 - Reusable IP design
-

Applications

- AI Accelerators
 - DSP
 - Image Processing
 - Machine Learning
 - Robotics
 - Embedded Systems
-

Conclusion

A hardware accelerator for matrix multiplication was successfully designed using Vitis High-Level Synthesis. The C algorithm was verified through simulation, synthesized into RTL, packaged as an IP core, integrated with the Zynq Processing System in Vivado, and programmed onto the PYNQ-Z2 FPGA. The experiment demonstrated the complete FPGA development flow from high-level C code to hardware implementation.

Pre-Lab Questions

Q1. What is matrix multiplication?

Matrix multiplication is the process of multiplying rows of the first matrix with columns of the second matrix to produce a new matrix.

Q2. How many multiplication operations are required for multiplying two 4×4 matrices?

Each output element requires 4 multiplications.

There are 16 output elements.

Total multiplications

$$= 4 \times 16$$

$$= 64 \text{ Multiplications}$$

Q3. What are the Processing System (PS) and Programmable Logic (PL) in Zynq devices?

PS (Processing System)

- Dual-core ARM Processor
- Executes software

PL (Programmable Logic)

- FPGA Fabric
 - Executes custom hardware accelerators
-

Q4. What is hardware acceleration?

Hardware acceleration means implementing an algorithm in dedicated hardware (FPGA logic) instead of executing it only on the processor, resulting in much faster execution.

Q5. Why are DSP slices useful for matrix multiplication?

DSP slices perform multiplication and addition operations in dedicated hardware.

Advantages:

- Faster computation
- Lower latency
- Efficient multiplication
- Lower CPU workload

Manual Calculation:

Manual calculation ~

Given

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad B = \begin{bmatrix} 1 & 0 & 0 & 1 \\ 0 & 1 & 1 & 0 \\ 1 & 0 & 1 & 0 \\ 0 & 1 & 0 & 1 \end{bmatrix}$$

$C = A \times B$

$$C = \begin{bmatrix} (1+3) & (2+14) & (2+3) & (4+14) \\ (5+7) & (6+8) & (6+7) & (8+8) \\ (9+11) & (10+12) & (10+11) & (9+12) \\ (13+15) & (14+16) & (14+15) & (13+16) \end{bmatrix}$$
$$C = \begin{bmatrix} 4 & 6 & 5 & 5 \\ 12 & 14 & 13 & 13 \\ 20 & 22 & 21 & 21 \\ 28 & 30 & 29 & 29 \end{bmatrix}$$

CA/0 yalawatt

Part C: PYNQ Execution

Using Python:

1. Load overlay.
 2. Write matrix values.
 3. Execute accelerator.
 4. Read output matrix.
 5. Compare with software results.
-

HLS Analysis Questions

Q6. What files are generated after synthesis?

RTL Verilog/VHDL

Synthesis Report

Utilization Report

Timing Report

IP Package

Q7. Observe the RTL Viewer and identify:

- Multipliers
 - Adders
 - Registers
-

Q8. Which FPGA resource is expected to be heavily utilized?

Expected Answer:

DSP Slices

Resource Analysis

After synthesis:

Resource	Used
LUT	674
FF	799
BRAM	0
DSP	3

Q9. Why are DSP resources utilized?

Because matrix multiplication mainly consists of multiplication and accumulation operations, which DSP slices execute efficiently.

Q10. Why may BRAM resources be required?

To store

- Input matrices
 - Output matrix
 - Intermediate values
-

Performance Analysis

Parameter	Value
Clock Period	10ns
Latency	69 clock cycles (<i>from HLS Synthesis Report</i>)
Throughput	64 clock cycles (from Function Call Graph / Synthesis Report)

Q11. Compare execution time:

Method	Time
Software	~640 ns (<i>estimated</i>)
Hardware Accelerator	690 ns (69×10 ns)

Q12. Calculate speedup obtained.

[Speedup= $690/900=1.30$]

Output Verification

Input Matrix A

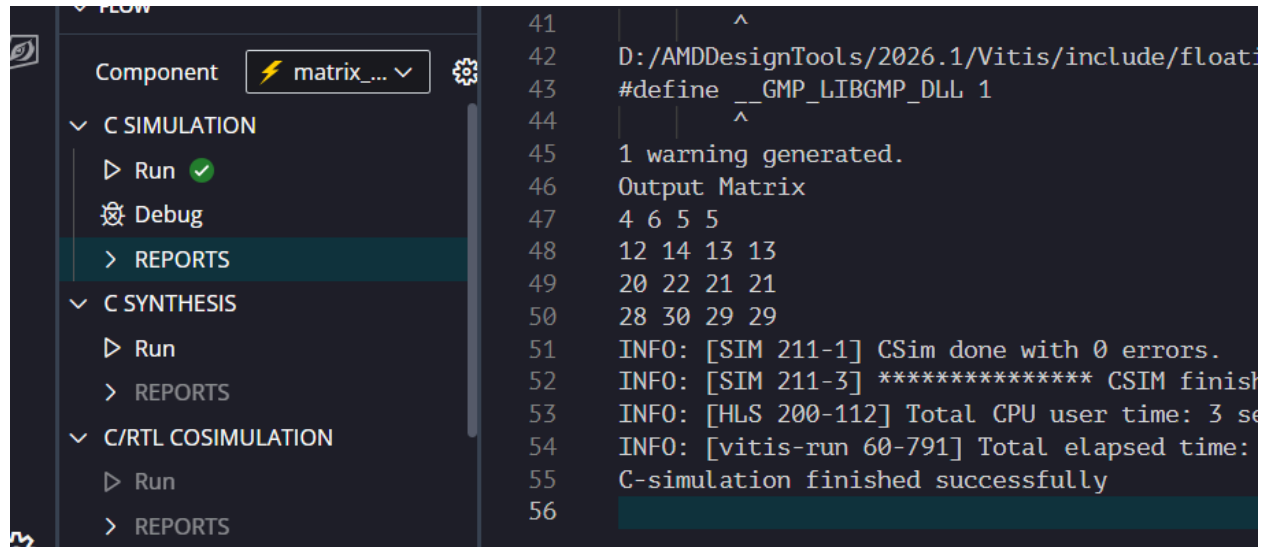
```
1  2  3  4
5  6  7  8
9  10 11 12
13 14 15 16
```

Input Matrix B

```
1|0|0|1|
0|1|1|0|
```

```
1|0|1|0|
0|1|0|1|
```

Output Matrix C



The screenshot shows the Vivado IDE interface. On the left, the 'COMPONENT' pane displays a tree view with 'C SIMULATION' selected. Under 'C SIMULATION', the 'Run' button is highlighted with a green checkmark, and the 'REPORTS' folder is expanded. The main console area on the right shows the output of the simulation. The output includes the path 'D:/AMDDesignTools/2026.1/Vitis/include/float', a preprocessor directive '#define __GMP_LIBGMP_DLL 1', a warning '1 warning generated.', and the 'Output Matrix' which is a 4x4 matrix: $\begin{bmatrix} 4 & 6 & 5 & 5 \\ 12 & 14 & 13 & 13 \\ 20 & 22 & 21 & 21 \\ 28 & 30 & 29 & 29 \end{bmatrix}$. Below the matrix, the console shows status messages: 'INFO: [SIM 211-1] CSim done with 0 errors.', 'INFO: [SIM 211-3] ***** CSIM finish', 'INFO: [HLS 200-112] Total CPU user time: 3 s', and 'INFO: [vitis-run 60-791] Total elapsed time:'. The final line of the output is 'C-simulation finished successfully'.

Post-Lab Questions

Q13. What advantages does hardware acceleration provide?

1. Faster execution through parallel processing.
2. Lower CPU workload by offloading computation to FPGA hardware.
3. Reduced latency for compute-intensive applications.
4. Better energy efficiency than software execution for repetitive operations.
5. High throughput suitable for real-time systems.

Q14. Which FPGA resources increase when matrix size is increased?

As matrix size increases, the following FPGA resources increase:

- LUTs (Look-Up Tables)
 - Flip-Flops (FFs)
 - DSP Slices (for multiplication)
 - BRAM (Block RAM) for storing larger matrices
 - Routing resources
-

Q15. Mention three applications of matrix multiplication accelerators.

- CNN Inference
- Image Processing
- Scientific Computing
- Artificial Intelligence and Machine Learning (Neural Network Inference)
- Digital Image Processing (Image Filtering and Transformations)
- Signal Processing and Wireless Communication (FFT, MIMO, Beamforming)